

MINI PROJECT – 2

Chakradhar Reddi Vitta

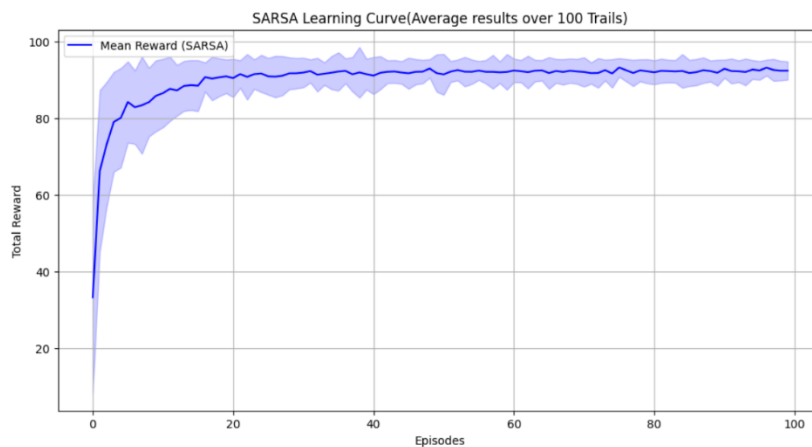
934-595-987

PART-A

In the part A, I implemented the code and compared different TD algorithms to solve the grid world problem, where agent reach the goal state by avoiding the fire and water states.

1.

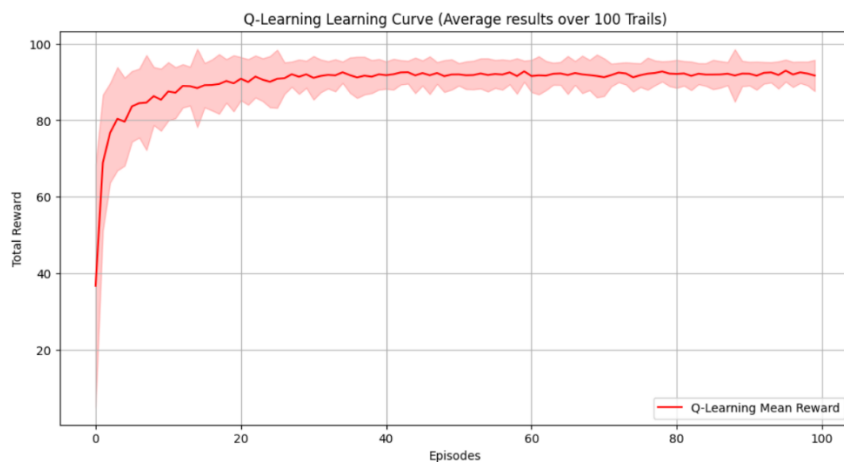
- greedy policy - starts with random actions, then prefers the highest Q-value action while still exploring 10% of the time.
- Parameters used in this are
Learning rate – 0.1
- Discount factor-0.95
- Exploration Rate-0.1
- Episodes- 100
- Trials -100
- It uses the On-policy learning where the performance is stable but slow convergence.
- SARSA updates Q-values based on the action chosen from the current policy, meaning it follows its own learning policy while training.
- Ensures stability and reduces noise in results.



2.

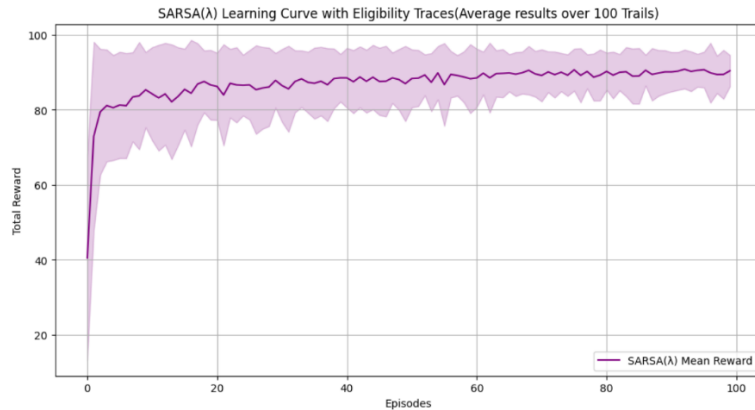
- greedy policy - starts with random actions, then increasingly selects the best Q-value action while still exploring.

- Parameters used in this are
Learning rate – 0.1
- Discount factor-0.95
- Exploration Rate-0.1
- Episodes- 100
- Trials -100
- It uses the On-policy learning, greedy updates where the performance is faster but unstable.
- Q-learning updates Q-values using the best next action, even if the current policy is different, leading to faster but sometimes unstable learning



3.

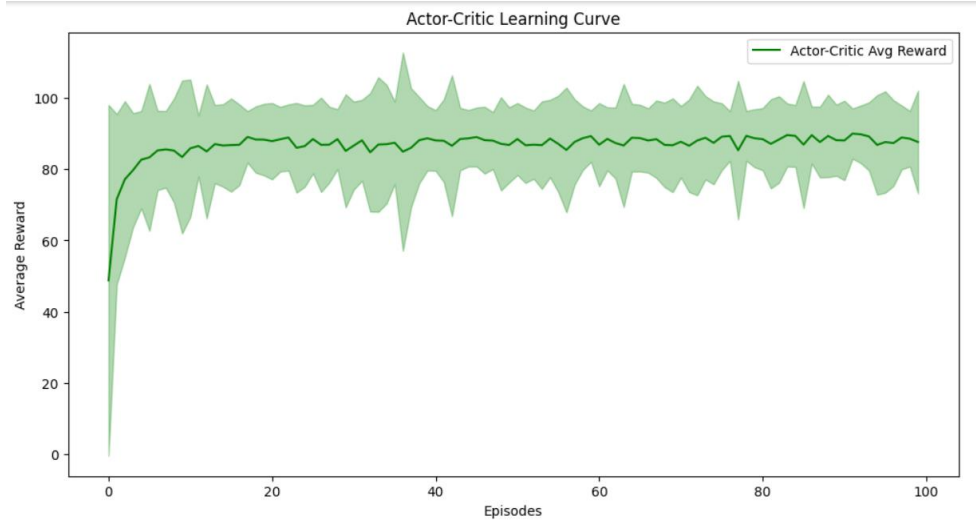
- greedy policy - same as SARSA, but with eligibility traces to update multiple past actions at once.
- Parameters used in this are
Learning rate – 0.1
- Discount factor-0.95
- Exploration Rate-0.1
- Episodes- 100
- Trials -100
- Eligibility Decay-0.9
- It uses the Eligibility traces where the performance is fastest learning but requires tuning of Eligibility decay.
- λ controls how much past states are updated; higher λ means more past actions are updated.



PART-B

1.

- Softmax policy - all actions start with nearly equal probability, then gradually shift toward higher-reward actions.
- Parameters used in this are
Learning rate(α -critic) – 0.1
- Learning rate(β -critic)-0.1
- Discount factor-0.95
- Exploration Rate-0.1
- Episodes- 100
- Trials -100
- Eligibility Decay-0.8
- Actor-Critic learns steadily but is sensitive to hyperparameter tuning.
- The initial policy used in Actor-Critic is a stochastic softmax policy. This means at the start of training, all actions have nearly equal probability of being selected. Over time, as the Actor updates its weights, the policy gradually shifts towards choosing higher-reward actions.
- Unlike SARSA and Q-learning, which store Q-values for each state-action pair, Actor-Critic uses function approximation to estimate policy and value functions. This is necessary for handling large state spaces, where storing a Q-table is inefficient. Instead, the policy is represented as a function that is updated gradually over time.



2.

- Each algorithm follows its respective initial policy - SARSA, Q-learning, and SARSA(λ) use ϵ -greedy, while Actor-Critic uses Softmax.
- Q-learning converges faster than SARSA but is unstable.
- SARSA(λ) learns the fastest among TD methods.
- Actor-Critic is competitive with SARSA(λ) but requires fine-tuning.
- Function approximation makes Actor-Critic suitable for larger environments.
- Standard deviation error bars show how much total rewards vary across trials, providing a measure of stability.
- The error bars in the comparison graph show the variability in total rewards over 100 trials.
- A smaller range means more stable learning (SARSA and SARSA(λ)).
- A wider range means more fluctuations (Q-learning and Actor-Critic).
- Actor-Critic's stability depends on how well the function approximation is tuned.

